# Ch. 13 Notes

## Additional Algorithms and List Forms

### Bubble sort

Repeatedly pass through the array swapping adjacent out-of-order pairs.

```
for (int i = 0; i < n - 1; ++i) {
    bool swapped = false;
    for (int j = 0; j + 1 < n - i; ++j)
        if (a[j] > a[j+1]) {
            std::swap(a[j], a[j+1]);
            swapped = true;
        }
    if (!swapped) break;   // early exit
         if sorted
}
```

$O(n^2)$ worst, $O(n)$ best (sorted). Stable, in place. **Never use in production** – insertion sort beats it in every dimension. Useful only as a teaching example and as a simple way to "almost-sort" partially-sorted data.

### Quickselect (find the $k$-th smallest)

Quicksort, but only recurse into the partition that contains $k$.

```
int quickselect(vector<int>& a, int lo,
    int hi, int k) {
    int p = lomutoPartition(a, lo, hi);
    if (p == k) return a[p];
    if (k < p) return quickselect(a, lo,
            p-1, k);
    else       return quickselect(a, p
        +1, hi,    k);
}
```

Expected $O(n)$; worst $O(n^2)$ (bad pivot). C++: `std::nth_element(first, nth, last)` – $O(n)$ expected, puts the $n$-th smallest at the `nth` position, with everything smaller before and larger after.

### Applications of quickselect

Median, top-$k$, percentile queries without a full sort. For streaming / huge $n$: combine with a heap of size $k$.

### Bucket sort

Assumes a roughly uniform distribution over a bounded range.

1. Split the range into $k$ buckets.
2. Distribute items into buckets by a key function.
3. Sort each bucket (insertion / `std::sort`).
4. Concatenate.

Average $O(n + k)$ if items distribute uniformly; worst $O(n^2)$ (all into one bucket).

### Bucket / counting / radix family

| use when | algo |
|---|---|
| small integer range | counting sort |
| uniform over $[0, 1)$ or similar | bucket sort |
| multi-digit / multi-byte keys | radix (LSD / MSD) |
| general / comparisons only | merge / quick / heap |

## List ADT vs. linked list

**List** = sequential positional collection ADT. One *implementation* is a linked list. Wrap the raw node-pointer code in a class with invariants (head, tail, size, no cycles).

```
template <class T>
class List {
    struct Node { T val; Node* next; };
    Node* head_ = nullptr;
    Node* tail_ = nullptr;
    std::size_t size_ = 0;
public:
    void push_front(T); void push_back(T
        );
    void pop_front();   void pop_back();
    std::size_t size() const { return
        size_; }
    // iterators, destructor, rule of 5
        ...
};
```

## Why wrap, why not just `std::list`

**Wrap own** when learning, when teaching, or when you need a special invariant (XOR list, intrusive list, lock-free). **Prefer `std::list`** in real code – it's been tuned, supports iterators, has the rule of 5 right. **But usually**: prefer `std::vector` unless profiling proves otherwise. Linked lists lose on cache locality, most of the time.

## Circular lists

Tail's `next` points to head (singly); plus head's `prev` points to tail (doubly).

- No null end – termination is **"came back to the start"**.
- Natural for round-robin schedulers, ring buffers, infinite playlists, Josephus problem.
- Traversal trap: forgetting the terminator and looping forever.

## Sentinel DLL (the clean variant)

Allocate a dummy `head` and `tail` (or a single sentinel that points to itself). Real nodes sit between.

- No empty-list special case.
- Insert / remove: always 4 pointer writes, never a nullptr check.
- Matches `std::list`'s internal shape.

## When *not* to reach for a linked list

- Random access / indexing by position (vector wins).
- Dense numeric data with tight loops (cache murder).
- Tiny elements where the pointer overhead dominates.
- When you need `sort` / `binary_search` – no random access, no `std::sort`.

## Top gotchas

- Using bubble sort in anything that ships.
- Quickselect with first-element pivot on sorted data – quadratic; shuffle or use random pivot.
- Bucket sort with skewed data – degenerates to one bucket's sort.
- Forgetting the rule of 5 in a hand-rolled `List` (copy / move / destructor leaks).
- Circular-list traversal with `while (n)` instead of `do {...} while (n != start)`.